

BIM and ReadSeal: Safe Early Reuse of Shared GPU Buffers Across Processes

Xingbo Hui, Xuetao Jiang, Guodong Ye, Ruochen Shao, Tianyi Wang, Sicheng Wu, Rui Zhou*, Qingguo Zhou*

School of Information Science and Engineering, Lanzhou University, Lanzhou, China

*Co-corresponding authors: zr@lzu.edu.cn, zhouqg@lzu.edu.cn

Abstract—Processes in a perception pipeline often share one large GPU tensor rather than giving each a private copy. The producer—Autonomous-driving and robotics software runs as many processes, such as ROS 2 nodes, and several of them often consume the same sensor data. As perception moves to the GPU, that data is increasingly one large tensor, such as a bird’s-eye-view (BEV) feature map, and ROS 2 has begun to support sharing it in GPU memory instead of copying it for each recipient. The producing process must then decide when it may to overwrite the shared buffer with the next frame. CUDA stream-only release reuses the buffer once all submitted GPU work has finished, so: too early, and a recipient that has not started can silently compute on the next frame. Full retention is safe but waits for every result, refusing new frames yet started can read the wrong frame; too late, and new frames are dropped while the GPU is idle. We observe idles. Our key observation is that a shared tensor has two lifetimes: its storage is needed only until the last read-completes operation that reads it, whereas its frame identity must last until every result is delivered. BEV-in-Memory (BIM) registers builds on this by registering every recipient process when a frame is published, so no recipient loses its frame protecting each one before it starts reading. Inside each recipient, ReadSeal analyzes storage sharing in the model graph to find the last operation that reads the shared tensor, and refuses to run a plan that no longer matches the executing it runs the plan only while the code, weights, or inputs and inputs still match. Recipients then finish on private intermediate results data while the buffer already holds the next frame. In an A100 pipeline with four early-reading recipients at 25–40 Hz, one BIM buffer delivers 1.31–1.50× as many correct, timely results as one-buffer full retention at 25–40 Hz. Relative to that baseline, mean latency increases by a buffer held until all results are ready, for 8–11 ms and 95th-percentile latency by 15–19 ms. At more mean latency; at 30 Hz, one BIM buffer also matches the timely delivery of full retention with two buffers and a tuned cap on outstanding requests. Derived read boundaries deliver as much it matches two such buffers with a tuned request cap. Automatically derived release points deliver as many results as hand-placed ones and are rebuilt automatically when the model changes. In ordered tests, a reader that submitted its work late computed on the next frame under stream-only release, but never under BIM follow model changes without manual updates.

Index Terms—GPU buffer reuse, zero-copy IPC, buffer lifetime, alias analysis, autonomous driving.

I. INTRODUCTION

Consider a driving stack built on Autoware or another framework based—Autonomous-driving stacks such as Autoware run as many processes on ROS 2, the Robot Operating System [1], [2], and several of them often consume the same data. A camera–LiDAR fusion node, for example, publishes a bird’s-eye-view (BEV) feature map, a top-down

grid of features around the vehicle, and separate processes for 3D detection, tracking, motion planning, and data recording read it on their own schedules (Fig. 1a). We call each such process a *recipient* and each component inside a recipient that reads the map a *reader*; BEVFusion, for example, feeds one BEV map to both a detection and a map-segmentation head [3]. Our evaluated map is a $1 \times 512 \times 180 \times 180$ fp32 tensor of 63.3 MiB [3]. Private copies for four recipients would add 256 MiB of device memory (64 MiB each after allocator rounding), and at 30 Hz the copying alone would read and write about 15 GiB/s. These costs grow with the number of recipients and frames in flight, and they matter most on automotive systems-on-chip, whose GPU shares DRAM with the rest of the stack [4]. Zero-copy inter-process communication (IPC) avoids them by letting recipients map the producer’s buffer [5], [6], and recent ROS 2 facilities extend this to GPU memory [7], [8]. The producer must then decide when it may overwrite that buffer, which we call a *slot*, with the next frame.

Two common release rules err in opposite directions (Section II). GPU work is asynchronous: other processes can wait only for work already submitted. CUDA stream-only release, stream-only for short, reuses the slot once all GPU work submitted so far has finished. A recipient still waiting in its queue has submitted nothing, so it can later compute on the next frame without any error. A detector would then report objects seen in frame $k+1$ under the timestamp of frame k ; at 20 Hz, a vehicle driving at 20 m/s moves a full meter between the two. Full retention holds the slot until every result is ready, as reference-counted zero-copy transports do when recipients keep their references until done [5], [6]. With one slot, frames that arrive during the hold are dropped. In our 20 Hz trace the hold barely exceeds the frame period, so full retention drops every other frame and leaves the GPU idle in between (Fig. 2).

Our key observation is that a shared tensor, which we call the *source*, has two lifetimes (Fig. 1b). Its *storage lifetime* ends when the last operation that reads it completes. Its *provenance lifetime* lasts until every result is delivered, because those results must still name the frame they came from, much as an essay keeps citing a library book after the book has gone back to the shelf. The two can differ widely. In the TransFusion detection head that reads our BEV map, only the first of 287 operations reads the source; the rest compute on intermediate results in private storage and are unaffected when the source is overwritten. A recipient therefore needs its *borrow*, its

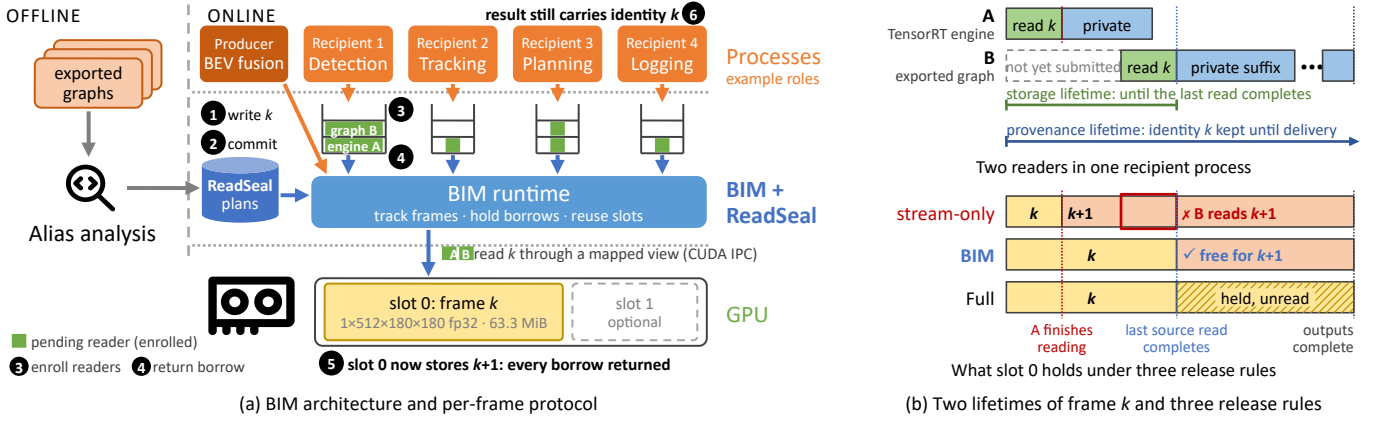


Fig. 1. **BIM at a glance.** (a) Offline, ReadSeal turns each exported model graph into a plan. Online, the producer writes frame k into a slot and commits it (❶, ❷); each recipient enrolls its readers before they submit GPU work (❸) and returns its borrow when their source reads complete (❹); once every borrow is returned, the producer may overwrite the slot (❺), while results still carry identity k (❻). (b) TensorRT engine A and exported graph B are, the two readers of the last recipient still holding a borrow, decide when the slot can be reused; time runs left to right. Stream-only release lets $k+1$ overwrite the slot before B reads it, full retention holds the slot until the outputs complete, and BIM reuses it after the last source read.

permission to read the slot, only until its last source read completes. Full retention, copying, and early reuse differ only in where the borrow ends: when the outputs are complete, when a private copy is complete, or at the last source read. All three must protect a recipient from the moment its frame is published, before it starts reading.

Ending the borrow at the last read is safe only if three questions can be answered. (1) Which operation reads last? *Views*, tensors that share the source’s storage, extend its reads, so the *read boundary* can come long after the source itself last appears (Fig. 3). (2) Who may still read? A reader that has not yet submitted GPU work leaves no event to wait on. (3) Is the analyzed program the one that runs? A boundary derived for one model version can be wrong for the next. Compilers answer the first question for a single program [9], [10]; here the producer sees neither the recipients’ programs nor the reads they have yet to submit.

We present BEV-in-Memory (BIM), a cross-process borrowing contract-protocol motivated by BEV sharing, and ReadSeal, which decides where each recipient’s borrow ends. The producer knows only which processes receive a frame, so BIM answers question (2) across processes: it registers every recipient’s borrow when the frame is published, before any recipient starts (Section III). Each recipient knows its own components, so ReadSeal answers the rest inside it: it enrolls all readers before any of them submits GPU work, derives each reader’s boundary by alias analysis of its model graph, and binds the plan to the code, model state, and input layout that execute (Section IV). BufferQueue and NvSciStream already let each consumer return a shared buffer on its own [11], [12], but they leave the application to decide at which point in its computation to return it and how to cover components that have not started reading. BIM and ReadSeal settle both: a borrow starts at publication and ends at a derived, checked last read.

A hand-placed boundary can fail in two ways. It can go stale: one added late read moves our detection head’s boundary from the first operation to the second-to-last (Fig. 3). Or it

can be wrong from the start: the first operation is the right boundary for the head, but in a ResNet-18 tail ~~the~~ the network after its first layers [13], a residual connection reads the source again at operation 12. A version check catches the first failure but not the second. ReadSeal avoids both, because it derives the boundary for each model version and refuses to run a plan that no longer matches execution. Our contributions are:

- **A two-lifetime view of shared GPU tensors** (Sections I–II). Separating storage from provenance shows when a buffer can be reused while results keep their frame identity, and ~~it~~ places full retention, copying, and early reuse on one axis.
- **Checked release at a derived last read** (Sections III–IV). BIM protects every recipient from publication onward, including readers that have not submitted work, and ReadSeal derives where each reader stops reading and rejects plans that no longer match execution.
- **An evaluation of where borrows should end** (Section V): early reuse against full retention, extra slots, and copying under load, and derived boundaries under ordered overwrites and model updates.

With early source reads, one BIM slot delivers $1.31\text{--}1.50\times$ the timely results of one-slot full retention at $25\text{--}40\text{ Hz}$ ~~and, Hz, for 8–11 ms more mean latency, and~~ at 30 Hz, Hz matches two full-retention slots with a tuned request cap while using one fewer 64 MiB buffer; mean latency rises by 8–11 ms. Early release also changes how a pool should be sized: given a second slot and no cap, BIM admits more work than the GPU can finish in time, and its timely results fall from 78.5 to 24.9 per second. Derived boundaries deliver as much as hand-placed ones and follow model changes automatically. Early release also changes pool sizing: a second slot then needs a request cap. When a recipient reads the source late, ~~early release gains nothing, and~~ per-recipient copies are the better choice if memory allows.

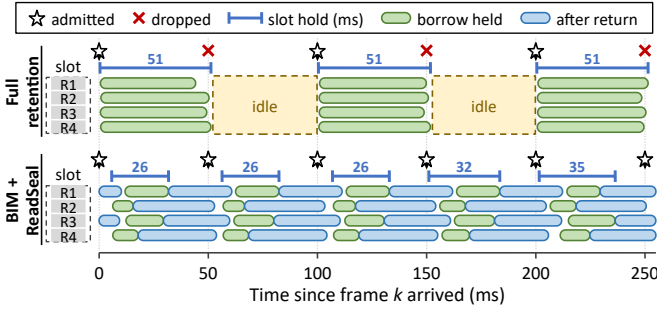


Fig. 2. **Full retention refuses frames while the GPU idles.** Measured one-slot trace at 20 Hz (repetition 0); R1–R4 are the recipients, and labels give slot hold times in ms. Full’s 51 ms hold just exceeds the 50 ms period, so every other frame is refused. Under BIM, each recipient’s work after its borrow return overlaps the next frame.

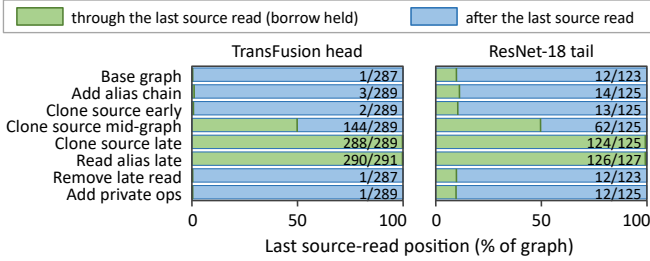


Fig. 3. **Model edits move the last source read.** Each bar is a graph snapshot that applies one edit to the base graph (Remove late read undoes Clone source late). Green marks the operations up to the last source read, during which the recipient holds its borrow; labels give that operation’s index/total operations. Add alias chain inserts two views, which renumber operations without moving the release point.

II. BACKGROUND AND MOTIVATION

Each release rule below ends a recipient’s borrow at a different point; Table I compares them.

GPU background. A *kernel* is a function executed on the GPU. A process places kernels in a *stream*, which executes them in order, and continues without waiting. A read ends when its kernel completes. An *event* marks a position in a stream and completes after the preceding work. PyTorch’s `record_stream` lets the caching allocator reuse a buffer only after the work already enqueued on the recorded streams has completed [14]. CUDA IPC lets another process map the same device buffer and wait on shared events. But an event can only mark work that is already enqueued, so no process can wait on reads that another process has not yet submitted.

Releasing too early fails silently. A rule that waits only for submitted work, such as `record_stream` or a read event, releases the slot once the other recipients’ reads complete, although a recipient still waiting in its queue has enqueued nothing. That recipient later computes a numerically valid result on frame $k+1$ while its request still names k , and no crash or error gives a downstream planner reason to distrust it. The same gap can open inside a-recipient-that combines a TensorRT engine (NVIDIA’s inference runtime) with a framework-graphone recipient whose readers run on different runtimes, each with its own completion signal. In

TABLE I
RELEASE POLICIES FOR FOUR RECIPIENTS. EXTRA MEMORY COUNTS SOURCE SLOTS AND INPUT COPIES BEYOND ONE SLOT; K IS THE REQUEST CAP (SECTION III). [†] MANUAL AND MANUAL-CHECKED (SECTION V-A). [‡] STREAM-ONLY RELEASE CANNOT WAIT FOR READERS THAT HAVE NOT SUBMITTED GPU WORK.

Policy	Extra memory	Slot reusable after	Read boundary	Wrong frame?
Stream-only	—	submitted reads	—	possible [‡]
Full retention	—	all outputs	—	no
Full, 2 slots, $K=8$	64 MiB	all outputs	—	no
Clone-on-accept	256 MiB	all copies	the copy	no
Hand-placed [†]	—	all last reads	by hand	no
BIM + ReadSeal	—	all last reads	derived	no

our running example (Fig. 1b), the two readers are TensorRT engine A (NVIDIA’s inference runtime), which signals an input-consumed event after reading its input [15], and exported graph B: ~~An adapter forwards a view to the graph without reading the source itself, which receives a view of the source from an adapter that does not read it.~~ Once the engine’s event completes, stream-only release makes the slot reusable although the graph has not submitted its reads; Section V-E tests this ordering. ROS 2’s CUDA buffer backend exposes the same choice: a subscriber’s read handle records a read event when it is destroyed, and the block is recycled once every handle is released [16]. Destroying the handle after the engine’s read gives stream-only release; holding it until the outputs complete gives full retention.

Holding the source through computation. Full retention is not work-conserving: its hold refuses frames while the GPU idles (Fig. 2). A second slot absorbs some of those arrivals but can admit more work than the GPU finishes before the deadline (Section V-C).

Copying the source. A private copy lets a recipient compute after the shared slot is reused. The copy itself reads the source, however, so the slot must stay protected while the recipient waits in its queue and until its copy completes. Our clone-on-accept policy therefore uses BIM’s recipient registration and ends each borrow at copy completion.

III. BIM: FRAME IDENTITY AND BUFFER REUSE

BIM separates memory, frame identity, and permission to read. The producer owns a pool of N reusable GPU slots on one machine. It writes frame k into a slot and attaches an identity that every result from that frame carries; the same slot can later hold frame $k+1$. A recipient holds one borrow per frame, covering all of its readers, such as engine A and graph B, including readers still waiting to submit work. We keep three terms apart: a *release rule* decides when a recipient’s borrow ends, the recipient then *returns* its borrow by clearing its borrow bit, and the slot becomes *reusable* when its last bit clears.

Descriptors and views. Recipients receive small descriptors over a control plane such as ROS 2 and map the resident tensor, here through PyTorch’s CUDA IPC support [17]. A descriptor carries the frame identity (producer instance and sequence

Algorithm 1 BIM’s publication and ReadSeal’s plan and invocation. $b_{s,j}$: borrow bit of recipient j on slot s ; U_j : readers of j ; T : binding tokens.

```

1: procedure PUBLISH(frame  $k$ )                                ▷ producer (BIM)
2:   if no slot is free or cap  $K$  has no room for  $k$ , drop  $k$ 
3:   write  $k$  into a free slot  $s$ ; commit: set  $b_{s,j}$  for all  $j$ 
4:   send each recipient a descriptor of  $(k, s)$ 
5: procedure BUILDPLAN(graph  $G$ )                                ▷ offline, per model version
6:   compute  $Q_r$  by (2) and  $\beta$  by (3)
7:   split  $G$  at  $\beta$ ; record readers  $U_j$  and tokens  $T$ 
8: procedure INVOKE(plan, frame  $k$ )                                ▷ recipient  $j$ 
9:   check  $T$ ; keep identity  $k$  for every output
10:  enroll every reader in  $U_j$  before any of them submits GPU work
11:  submit the stages in declared order, checking  $T$  at each entry
12:  on failed check: cancel unsubmitted reads, drain, publish nothing
13:  on each completed or cancelled read: if (4) holds, clear  $b_{s,j}$ 
14:  on completed outputs: publish them with identity  $k$ 

```

number), a storage locator, and how to read it: element type, shape, strides, and a checked range of addressable bytes. BIM rejects stale identities and incompatible interpretations.

Borrow bits and admission. A process-shared lock protects slot reservation, the frame sequence, and one borrow bit per recipient, set while that recipient may still read the slot. An arriving frame takes a slot whose bits are all clear; if none is free, the producer drops the frame instead of waiting. Once the frame is written, the producer *commits* it by setting every recipient’s bit before sending any descriptor. A recipient still waiting in its queue is therefore protected, because its bit was set before it could possibly read. Clearing the last bit makes the slot reusable without deallocating or unmapping it. If frame k is committed at time c_k and recipient j returns its borrow at time $t_{j,k}$, the slot is held for H_k . With one slot, a frame arriving at time a is admitted only if

$$a \geq c_k + H_k, \quad H_k = \max_j t_{j,k} - c_k. \quad (1)$$

Full retention sets $t_{j,k}$ to the time j ’s outputs complete, and ReadSeal to the time j ’s last source read completes (Section IV-B), so ReadSeal’s shorter hold admits more of the arriving frames. An optional request Each admitted frame gives every recipient one request, outstanding until its result reaches the output sink, the process that collects results. An optional cap K bounds the recipient requests outstanding between admission and receipt at the output sink; outstanding requests across the pipeline; a frame is admitted only if a slot is free and the cap has room for it.

IV. READSEAL: CHECKED BORROWING

ReadSeal prepares a *plan* once per model version and checks it each time a recipient processes a frame (Algorithm 1). The plan records where the graph stops reading shared storage, which components may read it, and which program version was analyzed. One execution of a plan is an *invocation*; its recipient returns the borrow when every planned source read has finished or was cancelled before starting.

A. Deriving the Read Boundary

The analysis runs on each graph, a fixed-shape, functional reader’s exported graph: a directed acyclic graph (DAG) of

operations with fixed tensor shapes. Only the storage lifetime needs analysis, because every result carries the frame identity attached at acceptance. Storage sharing propagates through views. A slice of the source is a view, so an operation on the slice still reads the source, whereas a convolution reads the source but writes fresh storage, so operations on its output do not.

Formally, let $S(v)$ be the set of shared roots whose storage a value v may alias, with $S(r) = \{r\}$ for the source r . For an operation o with inputs v_i and output v_o , let $\mathcal{A}_o$ index the inputs the output may alias and $\mathcal{R}_o$ the inputs o reads. Then

$$S(v_o) = \bigcup_{i \in \mathcal{A}_o} S(v_i), \quad (2)$$

$$Q_r = \{o : \exists i \in \mathcal{R}_o, r \in S(v_i)\},$$

where Q_r is the set of operations that may read the source. With the operations numbered $o_1, \dots, o_n$ in execution order, the read boundary is the position of the last one in Q_r ,

$$\beta = \max\{i : o_i \in Q_r\}, \quad (3)$$

so the *source-reading prefix*, $o_1, \dots, o_\beta$, includes all source reads, and the *private suffix*, $o_{\beta+1}, \dots, o_n$, computes the remaining results in private **intermediate** storage. If no operation reads the source, Q_r is empty and we set $\beta = 0$. Fig. 3 labels each **graph**-snapshot with β/n ; the base head has $\beta = 1$ and $n = 287$.

Operator summaries cover The sets $\mathcal{A}_o$ and $\mathcal{R}_o$ come from reviewed summaries of 42 reviewed-operator variants in ATen, PyTorch’s operator library, plus fixed-index tuple selection. Each summary counts every operand conservatively counts every input as read and treats an output as sharing lets an output share storage whenever the operator’s declared signature permits aliasing with an input it. Plan construction rejects unsupported mutation, opaque callbacks what it cannot analyze: unsupported in-place updates, calls into code it cannot inspect, and source views that escape-leave the graph. The implementation compiles the prefix and suffix into separate executable stages, and a CUDA event after the prefix marks the end of its source reads.

B. Enrolling Readers Before Submission

Protection works at two levels. BIM protects each recipient process by setting its borrow bit at commit. When that process accepts-accepts a frame, taking its request from the queue, ReadSeal registers, or *enrolls*, all of its readers before any of them submits GPU work. Each reader u marks its last source read with a completion event: A’s input-consumed event or the event after B’s prefix. Let U_j be the readers of recipient j . Then j returns its borrow for frame k at the first moment when

$$\forall u \in U_j : \text{done}_u \vee \text{cancelled}_u, \quad (4)$$

where the predicate done_u becomes true when u ’s completion event completes, and cancelled_u when u is cancelled before submission. Every enrolled reader takes part in this condition, including even one that has enqueued nothing: when A’s event completes, B’s read is still pending, so the bit stays set. Once all recipients have returned their borrows, the producer can

write the next frame while j 's **private**-suffix keeps computing results labeled k .

C. Binding Plans to Execution

At construction, ReadSeal captures the functions to run and copies the model's ~~parameters and buffers weights and other state, such as normalization statistics~~, once into plan-owned storage, so later edits by the caller cannot change a plan in use; the hand-placed baselines keep the same copy.

ReadSeal records *binding tokens* T : identifiers and metadata for code and ~~globals~~global variables, model storage and version, input layout, and compiled stages. At acceptance and at each stage entry it compares them with the values $\hat{T}$ for the execution about to run and proceeds only if $\hat{T} = T$, so other code, weights, or input layouts fail the check. A failed check cancels unsubmitted reads, drains started work, and publishes nothing. The application supplies the exported graph, each native engine's last-input-read event, and the component and output dependencies; new operators ~~require~~need reviewed summaries.

D. Correctness Conditions

The analysis must include every operation that can read the source. It assumes that operator summaries conservatively describe reads and storage sharing, kernels obey those summaries, and the graph executes in the analyzed order.

Proposition 1. *Under these assumptions, Q_r in Eq. (2) contains every operation that can read the shared source. No operation after the derived boundary β reads that source.*

Proof sketch. Initially $S(r) = \{r\}$. At each operation, the union in Eq. (2) preserves the shared roots of every input that its output may alias. Induction over graph order therefore includes every source alias in the corresponding $S(v)$. The read rule places every operation consuming such storage in Q_r ; by Eq. (3), none follows o_β . $\square$

The runtime further requires that each reader's completion event follow its source reads, that mappings stay valid, and that identities stay unique while referenced. Under these conditions, enrolling readers before submission (Algorithm 1), returning a borrow only when Eq. (4) holds, and overwriting only after all bits clear preserve the source until every reader finishes, while results keep the identity captured at acceptance.

V. EVALUATION

~~Section V-B measures what ending borrows at the last read gains under load and costs in latency; Section V-C compares one early-released slot with two fully retained ones; Section V-D adds jitter, a late reader, and copying; Section V-E tests safety. We measure what early reuse gains and costs under load (Sections V-B–V-D), whether it stays safe under reordering and model changes; and Section V-F measures what the checks cost (Section V-E), and what its checks cost (Section V-F).~~

A. Setup

Platform. We evaluate cross-process reuse on one NVIDIA A100 PCIe 40 GB GPU. The producer, four recipients, and an output sink are separate processes that communicate through multiprocessing queues and share storage and events through CUDA IPC. To isolate reuse and admission from feature extraction, the producer replays saved BEV features. The runtime uses PyTorch 2.1.2, CUDA 12.1, and TensorRT 10.3. Without NVIDIA's Multi-Process Service (MPS), recipient kernels take turns on the GPU rather than running concurrently. Records, validators, and analysis scripts are public.¹

Workload. Graph B is BEVFusion's functional TransFusion head [3], [18], exported with 287 operations. The producer publishes precomputed 63.3 MiB feature maps from three fixed nuScenes-mini frames [19]. Engine A runs the head's shared convolution, ReLU, and global average pooling in TensorRT, and an adapter forwards a view to the graph. This controlled composition gives each recipient two readers of the same source, each with its own completion event. Four recipients run identical compositions in the rate and slot sweeps; Section V-D varies their components. A pretrained ResNet-18 tail [13], the its residual stages and classifier with saved stem activations reading saved activations of the first layers as the source, serves in the correctness, model-change, and cost tests.

Policies. All policies share the pool, workloads, queues, and admission settings. ~~BIM-BIM~~ uses ReadSeal, *Full* retains the source until its outputs complete, and *Clone-on-accept* gives each recipient a private input copy when it accepts a request. Two hand-placed baselines answer different questions. *Manual* is an independently implemented runtime that enrolls both readers and checks code and model digests, so it tests early reuse apart from our code (Sections V-B and V-C). *Manual-checked* runs on BIM's runtime and binding checks and differs only in where the boundary comes from, so it isolates the effect of deriving the boundary (Section V-D).

Metrics. The producer schedules frames at rate λ , in Hz. Each frame should yield four *results*, one output bundle per recipient, so the offered load is 4λ results/s. A result is *timely* if it passes source-identity and reference-output checks and reaches the output sink, after device-to-host transfer and IPC, within 100 ms of the frame's planned arrival; Fig. 5c re-scores the slot runs at 80–200 ms. *Receipt latency* runs from planned arrival to sink receipt, and *slot hold time* from commit to the last borrow return. ~~Section V-D also reports the complete-frame timely fraction: the share of scheduled frames, refused and partially-admitted ones included, whose four results are all correct and timely.~~

Queues and statistics. In the rate and slot sweeps, each recipient permits at most two pending requests and four in flight. ~~When set, the cap K limits outstanding requests across the pipeline; A cap of $K=8$ accommodates two full groups of four requests.~~ The producer drops frames rather than blocking, so every policy sees the same offered load. These runs have 2 s of warm-up, 12 s of measured arrivals, and a complete

¹<https://github.com/assasin831/readseal-artifact/tree/ispa2026-r19>.

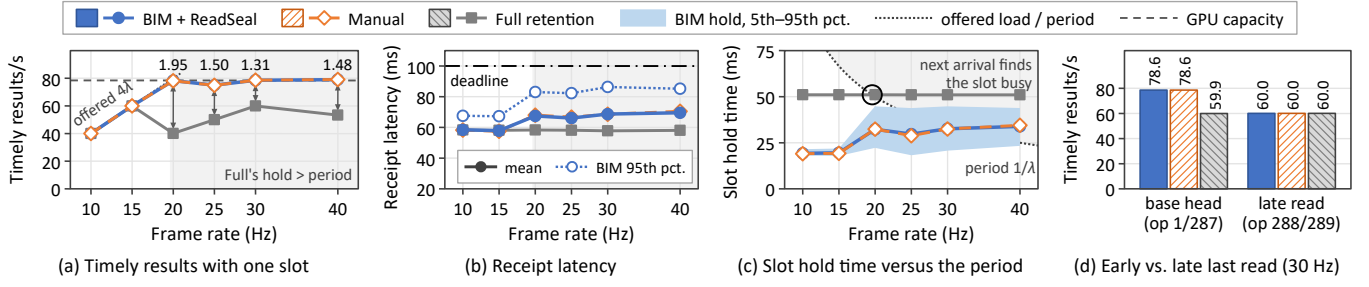


Fig. 4. **Early reuse raises one-slot delivery under load, at a latency cost.** One slot, no pipeline-wide cap K , six runs per condition and policy; each frame offers four recipient results. (a) Labels give BIM/Full ratios. (b) Mean receipt latency and BIM's mean per-run 95th percentile. (c) Mean hold and BIM's 5th–95th percentile; shading marks where Full's hold exceeds the frame period. (d) Moving the last read to the graph's end removes the gain.

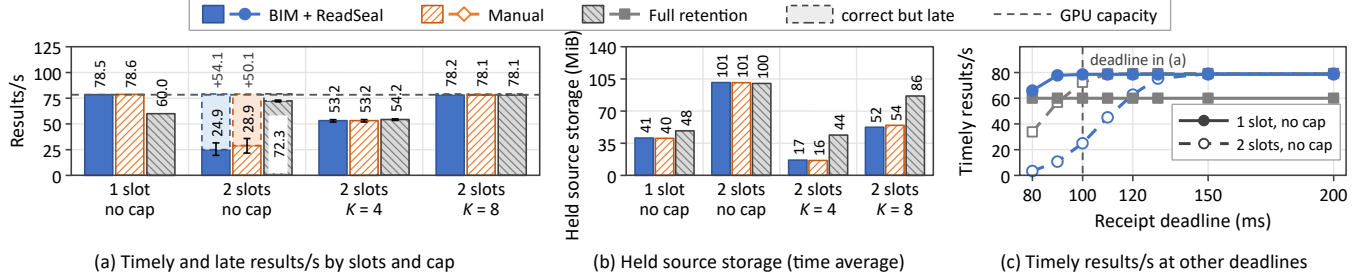


Fig. 5. **One BIM slot matches full retention with two slots and a request cap** (30 Hz; six runs per condition and policy). K counts outstanding recipient requests. (a) Solid: timely results; dashed: correct but late; whiskers: 95% intervals. (b) Time-averaged source storage held by unreturned borrows. (c) The same runs re-scored at other deadlines.

drain, with condition and method order randomized and six paired repetitions per condition; brackets give pointwise 95% intervals from 20,000 paired whole-run bootstrap draws. The two campaigns of Section V-D use queues of eight, 10 s of warm-up, 30 s of measurement, six paired runs, and pointwise 95% t intervals; each campaign is analyzed on its own.

B. Early Reuse Raises Delivery Under Load

Early reuse pays off once the frame period drops below full retention's hold time (Fig. 4a). Because the recipients take turns on the GPU and Full's 51 ms hold spans nearly all of their computation, the GPU can finish at most about four results per 51 ms, or 78 results/s (dashed line). Both policies keep up at 10 and 15 Hz. From 20 Hz on, the offered load reaches this capacity, so each frame Full refuses leaves GPU time unused. At 20 Hz, where Full accepts only every other frame, BIM delivers $1.95\times$ Full's timely results, and at 25–40 Hz it delivers $1.31\text{--}1.50\times$ as many, running near ~~the GPU's~~ capacity. BIM stays within 0.2 results/s of Manual at every rate.

The gain comes from a shorter hold: Full holds the slot for about 51 ms, ~~nearly the entire computation of the four recipients,~~ whereas BIM returns it after ~~their~~ the source reads, in 30–34 ms (Fig. 4c). The hold is not shorter still because the recipients take turns on the GPU: each head reads the source in its first operation, but the last recipient must wait for its turn before that operation runs. The shorter hold lets the next frame enter while private computation continues, which fills the idle gaps of Fig. 2. The dip at 25 Hz comes from periodic arrivals: the 40 ms period is shorter than BIM's longer holds

(95th percentile 44 ms), so some frames find the slot busy, and each refusal leaves it waiting a full period for the next arrival, which comes sooner at 30 and 40 Hz.

The extra admitted work queues behind other recipients on the GPU, which costs latency. At 20–40 Hz, BIM's mean latency is 8–11 ms higher than Full's and its per-run 95th percentile 15–19 ms higher (Fig. 4b). At 30 Hz BIM's delivery lead holds for deadlines from 100 to 200 ms and narrows at 80 ms, where more of this queued work arrives late (Fig. 5c).

The gain also depends on where the last read falls. Moving it to operation 288 of 289 (Fig. 3's late clone) leaves almost no private computation to overlap. BIM's hold then reaches Full's, and all three policies deliver 60.0 results/s at 30 Hz (Fig. 4d).

C. One Slot Matches Two, and Two Need a Cap

A second slot is the usual remedy for full retention's refusals (Fig. 5a). At 30 Hz, two-slot Full with $K=8$ delivers 78.1 results/s, matching one-slot BIM's 78.5, so BIM reaches the same delivery with one fewer 64 MiB source buffer. The second slot also makes the cap decisive: $K=4$ holds every policy to 53–54 results/s, and without a cap work misses the deadline.

With one slot, the hold time in Eq. (1) paces BIM's admissions. A second slot removes that pacing and lets requests queue before the GPU can start them; as with bufferbloat in networks, the longer queue adds delay but not throughput. Without a pipeline-wide cap, two-slot BIM delivers only 24.9 timely results/s, and another 54.1 results/s arrive correct but late. Full's longer hold still throttles its arrivals, so it delivers 72.3. Manual shows the same drop as BIM, and looser deadlines recover the delayed results (Fig. 5c).

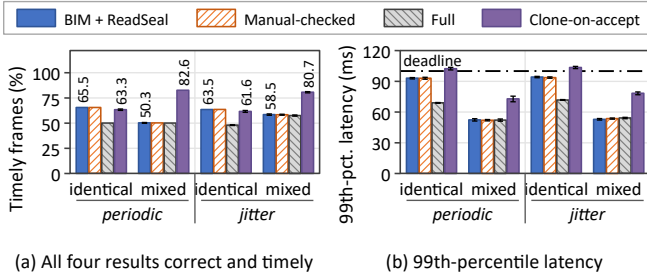


Fig. 6. **Complete-frame delivery and tail latency with jitter and mixed recipients** (30 Hz, one slot, no pipeline-wide cap; six runs per bar; whiskers: 95% t intervals). (a) Share of the 900 scheduled frames per run whose four results are all correct and timely. (b) Per-run 99th-percentile latency of frames whose four results are all correct. *Identical*: four copies of the composition; *mixed*: four component profiles, one reading late.

At $K=8$, all three policies reach similar delivery across the two-slot rate sweep. BIM still returns borrows sooner, so **the source storage held by outstanding borrows averages 52.3 MiB against Full’s 86.4 MiB** (Fig. 5b). Returning a borrow makes a slot reusable without deallocating it, so **the device peak is the same under every policy** every policy has the same device peak: 5,078 MiB with one slot and 5,142 MiB with two.

D. Jitter, Mixed Recipients, and Copying

This study measures complete frames rather than individual recipient results. Instead of individual results, this study reports the complete-frame timely fraction: the share of scheduled frames, refused and partially admitted ones included, whose four results are all correct and timely (Fig. 6). **A frame is timely only if all four results are correct and timely.** At 30 Hz with one slot and no pipeline-wide cap, arrivals are periodic or displaced by up to 0.8 of a period (*jitter*). Recipients are either *identical* copies of the composition or *mixed*: the native engine alone, the exported head alone, their composition, and a composition that reads the source late.

With identical recipients, BIM completes 65.5% of frames on time under periodic arrivals and 63.5% with jitter, about 15.5 percentage points above Full in both cases. With mixed recipients, the late reader holds the slot for all four, and BIM stays less than one point above Full (Fig. 6a). Jitter raises mixed-recipient delivery from 50.3% to 58.5% for BIM and from 50.0% to 57.5% for Full, consistent with irregular gaps letting more frames past a hold that otherwise refuses every second periodic arrival. Across this study, including 12 all-late runs, Manual-checked stays within 0.13 percentage points of BIM: with the runtime and checks held fixed, deriving the boundary costs nothing in delivery.

Copying helps where early reuse cannot. Clone-on-accept returns its borrow when the copy completes, even if computation reads the private copy much later. With mixed recipients it completes 22–32 percentage points more frames on time than BIM, at a 256 MiB higher sampled device peak and 20–25 ms more 99th-percentile latency; with identical early-reading recipients it completes about two points fewer.

A separate 168-run study **returns to recipient-level measurement, counting individual timely results rather than**

complete frames compares BIM with cloning by the per-result metric of Sections V-B and V-C, **timely results/s**. With early readers, BIM delivers 0.4–0.9 results/s more than cloning at 20, 30, and 40 Hz, while cloning leads by 2.75 [2.47, 3.03] at 25 Hz, where BIM shows its admission dip. When all readers read late at 30 Hz, cloning delivers 76.4 results/s against BIM’s 60.0, a paired gain of 16.39 [16.06, 16.72], but it raises the mean per-run 99th-percentile latency from 68.7 to 101.1 ms and the sampled device peak from 5,078 to 5,334 MiB.

E. Safety Across Reordering and Model Changes

We first reorder reads on purpose. In six controlled tests, engine A’s input-consumed event completes before graph B submits its reads. Under stream-only release, the graph’s outputs matched the replacement frame in every test, because recording stream use cannot cover reads that have not been submitted [14]; under BIM and Manual, they matched the admitted frame. Under load, each of the 109,592 results delivered by BIM, Manual, and Full in the 144 rate and late-read runs carried its admitted identity and matched the reference outputs.

We then change the models. **Across In 96 ordered-overwrite cases** (eight graph snapshots per model, three inputs, and two methods, **regenerated boundaries kept all 96 ordered-overwrite cases exact against unsplit references, and binding**), **the source was overwritten once the regenerated boundary had completed, and every output still matched the unsplit model’s exactly.** Binding validation rejected each of 22 attempted substitutions of plan, code, globals, model state, or input before the affected stage ran.

Finally, we replay 16 controlled transitions: 14 graph edits, one **batch-normalization folding** **folding of batch normalization into the preceding convolution**, and a public ResNet-18 to ResNet-50 substitution, whose binding check runs on the CPU. The runtime release point moves in 13 of them; folding keeps the head’s first source read, and the two alias-chain edits only renumber operations (Fig. 3). Binding checks reject every outdated hand-placed plan, so a manual deployment stops until someone refreshes its plan. ReadSeal rebuilds the updated plans in 0.08–0.21 s on the CPU, with no source-reader or boundary annotations. In the mixed-recipient campaign, adding a late read **likewise** made Manual-checked’s old boundary fail its check until **the boundary it** was moved, after which all 24 update cases gave exact outputs.

F. Execution Cost and Coverage

An isolated-process test times recipient acceptance to output dispatch (8 process repetitions of 50 randomized invocations per method; three inputs; base and late variants of both models). ReadSeal’s checks add 0.8–0.9 ms per invocation, mostly at acceptance and stage submission: 8% over Manual for the base head (11.8 ms against 10.9 ms) and 22–23% for the lighter ResNet tail. In the pipelines above this cost overlaps GPU work; even at 10 and 15 Hz, BIM’s mean receipt latency is within 0.4 ms of Manual’s. A static survey of **the tails of** 14 torchvision **tails-models** accepts nine, with boundaries in their first 1–12 operations, and rejects five for unsupported operators.

VI. DISCUSSION

Choosing a release policy. ~~With Early reuse fits systems whose memory capacity and bandwidth are too scarce for per-recipient copies: with early reads, one BIM slot matched two full-retention-fully retained slots with a tuned cap and delivered about as much as per-recipient-copies without their 256 MiB. With a late read, early reuse gains nothing (Fig. 4d), and copying completed 22–32 points more mixed-recipient frames on time than BIM, at 20–25 ms more tail-latency.~~ Where memory is plentiful, extra slots or copies are simpler; ~~early reuse fits systems whose memory capacity and bandwidth are too scarce for per-recipient copies, and when a read falls late (Fig. 4d), copies deliver more.~~ Pool sizing changes too: one slot paced BIM’s admissions by itself, whereas two slots needed a request cap.

Portability. BIM needs cross-process storage mapping and completion events that cover each reader’s source access. ROS 2’s CUDA buffer read handles [16] and NvSciStream’s per-consumer packet release [12] are natural integration points: a port would create each recipient’s hold before the frame is published, release it on ReadSeal’s completion event, and validate the platform’s ordering guarantees.

Limitations. Our measurements come from one A100 without MPS, where recipients take turns on the GPU, and from replayed features rather than a ROS 2 stack with its BEV front end. On this 40 GB card, the 64 MiB slot that BIM saves is small beside a 5 GiB device peak; the saving matters where the GPU shares DRAM with the rest of the system, which we have not measured. The six ordered tests show that stream-only release can serve the wrong frame, not how often it does so under load. The prototype runs declared topologies with single-stream stages; a topology change requires a new plan. A slow recipient holds the slot for everyone, as the late reader did in Section V-D, and a crashed one blocks reuse until recovery confirms that its GPU work has stopped.

VII. RELATED WORK

Zero-copy robotics middleware. iceoryx and Agnocast give ROS 2 subscribers direct access to publisher-owned shared memory [5], [6]. For GPU data, NITROS keeps its zero-copy path within one process [7], and ROS 2’s `rosidl::Buffer` moves tensor bytes between processes through CUDA IPC [8]. These systems decide how a buffer is shared but leave open when it may be reused while a subscriber has yet to read it.

Release fences. Android’s `BufferQueue` lets a consumer release a pooled buffer with a fence while still reading it [11], and NvSciStream returns a multicast packet to its pool once every consumer has released it [12]. BIM’s borrow bits follow the same per-consumer protocol, and ReadSeal adds where the fence goes, deriving from each recipient’s model where its reads end.

Asynchronous buffer protocols. Simpson’s four-slot mechanism separates concurrent reads and writes across buffers, and Kopetz and Reisinger’s NBW protocol detects overlapping writes and also provides a multibuffer variant [20], [21]. Cyclical asynchronous buffers let readers acquire the latest

complete value without blocking the writer [22]. BIM instead preserves the particular frame already assigned to each recipient, including while its GPU reads are unsubmitted.

Last use and safe reclamation. Compilers and runtimes reuse a buffer after its last use, as in TVM’s static memory planning [9], and Rust’s non-lexical lifetimes end a borrow there [10]; both see every use in one program, in program order. Task runtimes such as Legion and StarPU know future accesses because tasks declare their data when submitted [23], [24]. BIM’s recipients are separate processes that declare nothing to the producer. Safe memory reclamation, such as read-copy-update (RCU) grace periods [25], reuses memory once no reader can hold it, but protects only readers that have announced themselves. In RCU terms, BIM’s commit opens every recipient’s read-side section on its behalf, and ReadSeal closes it at the ~~derived~~ last read. ReadSeal’s bindings, like PyTorch 2’s guards [26], reject execution whose assumptions changed.

VIII. CONCLUSION

When several processes read one GPU buffer, the producer can overwrite it as soon as the last read completes, often long before the last result is ready, provided it knows which readers are still to come and where each one stops reading. BIM registers every recipient when a frame is published, and ReadSeal derives each reader’s last source read from its model graph and refuses to run a plan that no longer matches execution. Where a borrow should end depends on where that last read falls and on how much memory is free. In our A100 pipeline, where the detection head reads the shared map only in its first operation, ending borrows there let one slot deliver 1.31–1.50× the timely results of one-slot full retention at 25–40 Hz and ~~match two fully retained slots~~ at 30 Hz, ~~as many as two full-retention-slots with a tuned request cap, for 8–11 ms more mean-latency.~~ When a read falls late, the gain disappears, and per-recipient copies win if memory allows. Derived boundaries delivered as much as hand-placed ones and, unlike them, were rebuilt automatically for every model edit we replayed. A small static survey suggests that early reads are common: all nine torchvision tails that ReadSeal accepted stop reading their input within 12 operations.

Nothing in the contract is specific to BEV maps; it applies wherever several processes consume one large GPU tensor. Next, we will integrate BIM with ROS 2’s CUDA buffer backend and NvSciStream, measure BIM and its checks on embedded platforms whose GPU shares DRAM with the CPU, support recipients that run concurrently, and use each reader’s derived boundary to decide ~~recipient-by-recipient~~, whether to borrow or copy.

REFERENCES

- [1] Autoware Foundation, “Autoware: Open-source software for autonomous driving,” <https://autoware.org/>, 2024.
- [2] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Sci. Robot.*, vol. 7, no. 66, p. eabm6074, 2022.

- [3] Z. Liu *et al.*, “BEVFusion: Multi-task multi-sensor fusion with unified bird’s-eye view representation,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023, pp. 2774–2781.
- [4] NVIDIA, “CUDA for Tegra application note (CUDA 13.0),” <https://docs.nvidia.com/cuda/archive/13.0.0/cuda-for-tegra-appnote/index.html>, 2025.
- [5] Eclipse Foundation, “Eclipse iceoryx: True zero-copy inter-process communication,” <https://github.com/eclipse-iceoryx/iceoryx>.
- [6] T. Ishikawa-Aso and S. Kato, “ROS 2 Agnocast: Supporting unsized message types for true zero-copy publish/subscribe IPC,” in *Proc. 28th Int. Symp. Real-Time Distrib. Comput. (ISORC)*, 2025, pp. 1–10.
- [7] NVIDIA, “Isaac ROS NITROS: NVIDIA Isaac transport for ROS,” https://nvidia-isaac-ros.github.io/repositories_and_packages/isaac_ros_nitros/isaac_ros_nitros/index.html, 2026.
- [8] Open Robotics, “About `rosidl::Buffer` backends (ROS 2 Lyrical documentation),” <https://docs.ros.org/en/lyrical/Concepts/Intermediate/About-Buffer-Backends.html>, 2026.
- [9] T. Chen *et al.*, “TVM: An automated end-to-end optimizing compiler for deep learning,” in *Proc. 13th USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*, 2018, pp. 578–594.
- [10] N. Matsakis, “RFC 2094: Non-lexical lifetimes,” <https://github.com/rust-lang/rfcs/blob/master/text/2094-nll.md>, 2017.
- [11] Android Open Source Project, “BufferQueue and Gralloc,” <https://source.android.com/docs/core/graphics/arch-bq-gralloc>.
- [12] NVIDIA, “NvSciStream APIs (NVIDIA DRIVE OS 6.0 Linux SDK API reference),” https://developer.nvidia.com/docs/drive/drive-os/6.0.8/public/drive-os-linux-sdk/api_reference/group__nvsci__stream__apis.html.
- [13] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.
- [14] PyTorch Contributors, “`torch.Tensor.record_stream` (PyTorch 2.1 documentation),” https://pytorch.org/docs/2.1/generated/torch.Tensor.record_stream.html, 2023.
- [15] NVIDIA, “TensorRT 10.3: Execution context and input-consumed event contracts,” <https://github.com/NVIDIA/TensorRT/blob/release/10.3/include/NvInferRuntime.h>, 2024.
- [16] Open Robotics, “`cuda_buffer_backend` (ROS 2 `rosidl_buffer_backends`),” https://github.com/ros2/rosidl_buffer_backends, 2026.
- [17] PyTorch Contributors, “Multiprocessing package: Sharing CUDA tensors (PyTorch documentation),” <https://docs.pytorch.org/docs/stable/multiprocessing.html>.
- [18] X. Bai *et al.*, “TransFusion: Robust LiDAR-camera fusion for 3D object detection with transformers,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 1090–1099.
- [19] H. Caesar *et al.*, “nuScenes: A multimodal dataset for autonomous driving,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 11 621–11 631.
- [20] H. R. Simpson, “Four-slot fully asynchronous communication mechanism,” *IEE Proceedings E (Computers and Digital Techniques)*, vol. 137, no. 1, pp. 17–30, 1990.
- [21] H. Kopetz and J. Reisinger, “The non-blocking write protocol NBW: A solution to a real-time synchronization problem,” in *Proc. IEEE Real-Time Systems Symposium*, 1993, pp. 131–137.
- [22] G. Buttazzo, “Why real-time computing?” *Automazione e Strumentazione*, pp. 82–88, Feb. 2007, <https://retis.santannapisa.it/~giorgio/paps/2007/automazione07.pdf>.
- [23] M. Bauer, S. Treichler, E. Slaughter, and A. Aiken, “Legion: Expressing locality and independence with logical regions,” in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC)*, 2012, pp. 1–11.
- [24] C. Augonnet, S. Thibault, R. Namyst, and P.-A. Wacrenier, “StarPU: A unified platform for task scheduling on heterogeneous multicore architectures,” *Concurrency Comput.: Pract. Exper.*, vol. 23, no. 2, pp. 187–198, 2011.
- [25] P. E. McKenney and J. D. Slingwine, “Read-copy update: Using execution history to solve concurrency problems,” in *Proc. Parallel Distrib. Comput. Syst. (PDCS)*, Oct. 1998, pp. 509–518.
- [26] J. Ansel *et al.*, “PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation,” in *Proc. ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, vol. 2, 2024, pp. 929–947.